SOLUTION

Flag Verification Function:

The flags aren't hardcoded anywhere. The encrypted flags are present in the FlagVerification function. The players get the corresponding AES key of a flag when they solve a challenge. When they submit the key, they get the flag i.e., the clue.

Flags and corresponding keys

Challenge	Flag	AES Key
1	Island{The_Woman_with_the_Tight_Bun_was_not_in_the_Cell}	22ea0d77dd5d20e67680f7457d19c7ff479e43dfde865c623355cda61092fa85
2	Island{The_Odorless_Chemical_was_not_used_in_the_CCTV_Hub}	e85466670e29abc356d3f55c4a0f997d02caa1fd2ee5c7a2be1934d7a30fffc3
3	Island{The_Man_with_the_Silk_Handkerchief_was_in_the_Infirmary}	103f2312b591ff00d8bcf8673be282192a9720bf7d3ae6b6d4cb8a2ced46d1e1
4	Island{The_Silver-Haired_Politician_did_not_use_the_Black_Wire}	40b52d912753427421fc86902cfe16bff01be2353a06c4cdfe1a37293f70e7f0
5	Island{The_Braided_Linen_Cord_was_found_in_the_Cell}	4d82efb5ff0432fbc99992402fd03da5fbb45da4ab52b2b3d043c633f7347154
6	Island{The_Woman_with_the_Tight_Bun_carried_the_Black_Wire}	67a2efb4bae4c0b6cb559162971197bb765c9aad93a02593d559a089a81c3a94

1. DISGUISE **[Easy]**

   Want the key? Disguise as FRIEND_OF_KILLER. Maybe an imposter will give you the key!

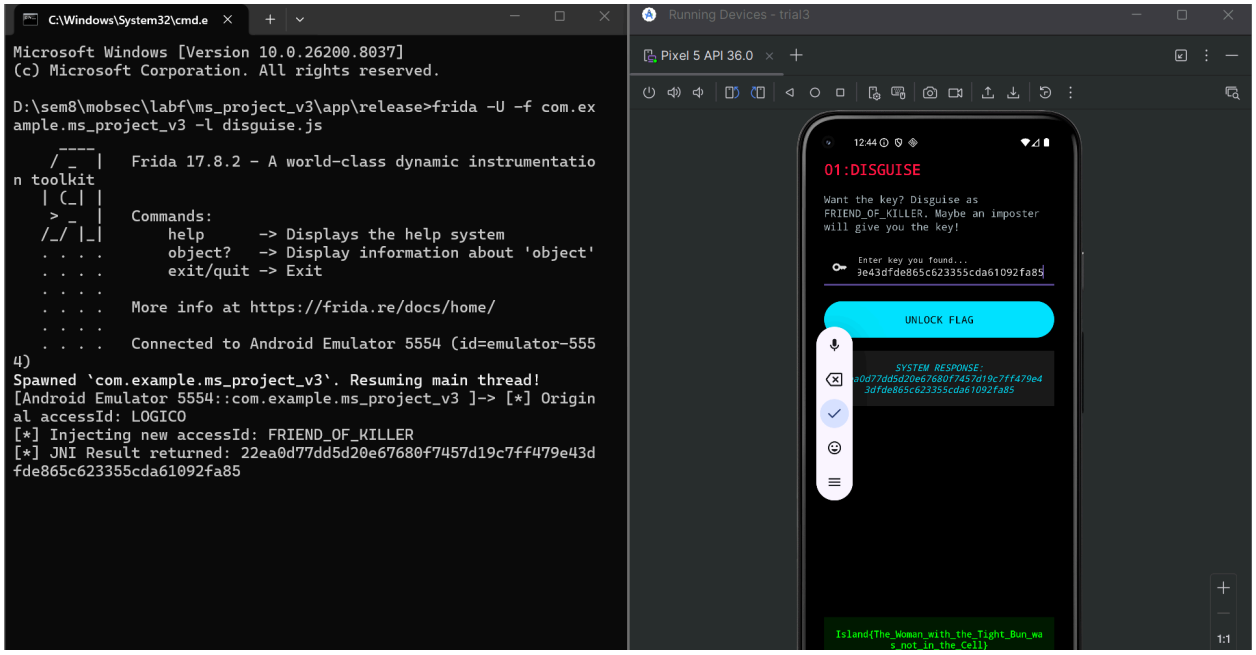
Solution:

```
String jniResult = getAttendanceRecord( accessId: "YOUR_ID");
```

Try using this hooked code

Hook this function **getAttendanceRecord("FRIEND_OF_KILLER")** using frida.

```
Java.perform(function () {
    var HookerActivity = Java.use('com.example.ms_project_v3.HookerActivity');
    HookerActivity.getAttendanceRecord.implementation = function (accessId) {
        console.log("[*] Original accessId: " + accessId);
        var newId = "FRIEND_OF_KILLER";
        console.log("[*] Injecting new accessId: " + newId);
        var result = this.getAttendanceRecord(newId);
        console.log("[*] JNI Result returned: " + result);
        return result;
    };
});
```



2. VIGILANTE **[Medium]**

   Try to intercept the communication between app and vigilante. You somehow have to prove yourself as the original app as well

Solution

```
    private void fetchPayload() {
        TextView textView = (TextView) findViewById(R.id.tvMessageDisplay);
        textView.setText("Letter received...");
        textView.setVisibility(0);
        Volley.newRequestQueue(this).add(new Request<byte[]>(0, "https://murdle-app.praveshini.xyz/", new Res
            @Override // com.android.volley.Response.ErrorListener
            public final void onErrorResponse(VolleyError volleyError) {
                this.f$0.m138lambda$fetchPayload$2$comexamplems_project_v3NetworkActivity(volleyError);
            }
        }) { // from class: com.example.ms_project_v3.NetworkActivity.1
            @Override // com.android.volley.Request
            public Map<String, String> getHeaders() {
                HashMap map = new HashMap();
                map.put("X-Murdle-Auth", NetworkActivity.this.getDecodedKey());
                map.put("User-Agent", "Murdle-Mobile-Agent-v3");
                return map;
            }
```

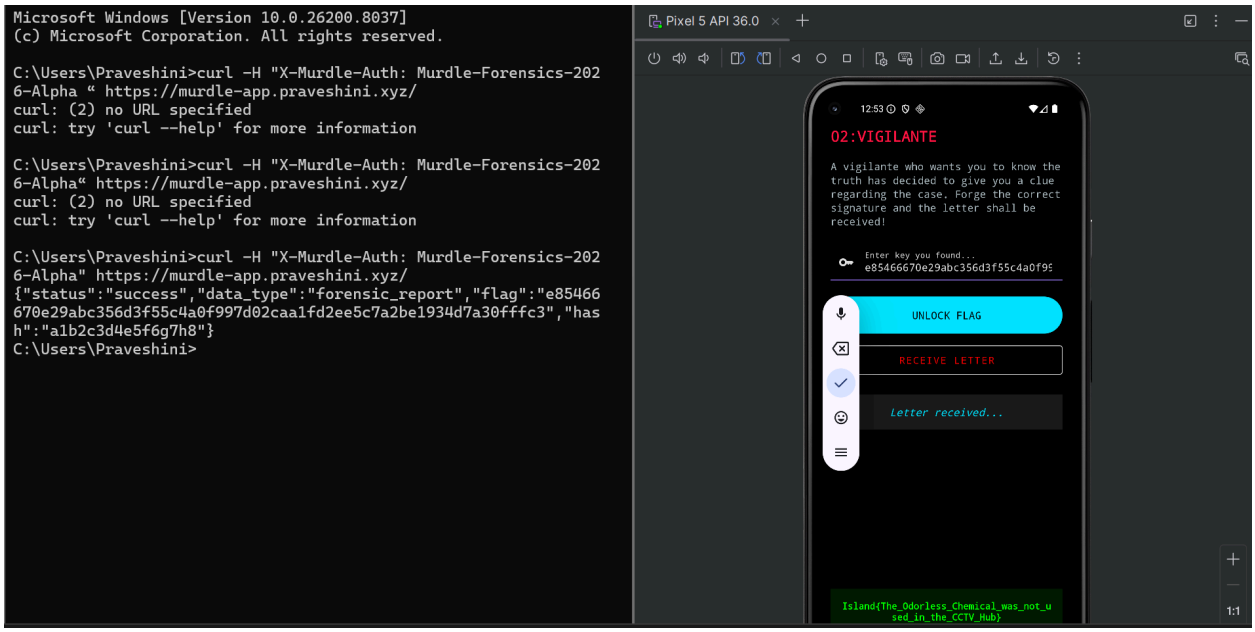
The solution exists in this part of the code. The domain https://murdle-app.praveshini.xyz/ provides the correct key if connected using header

“X-Murdle-Auth”: “Murdle-Forensics-2026-Alpha”

Solution (1) - Intercept the connection between app and domain using burp. This will reveal the key

Solution (2) - curl command using correct headers will also return the key

curl -H "X-Murdle-Auth: Murdle-Forensics-2026-Alpha “ https://murdle-app.praveshini.xyz/



3. THE WHISTLEBLOWER’S RANSOM **[Medium]**

   A whistleblower has intercepted the killer's GPS coordinates, but he’s gone into hiding. He demands a massive transfer of BoneCoin (BNE) to fund his disappearance (($2,147,483,647 in USD)

   The current market price is $1,000,000 USD per 1 BNE.

   You only have $500 USD in your account. You can't afford even a fraction of a coin, unless you can trigger a Negative Invoice.

   [Look where the associated transaction function is. Integer is 32 bits]

Solution:

Integer overflow

The application attempts to validate a transaction by checking if (Price * Quantity) <= Balance.

However, it performs this calculation in Native library(whistleblower_lib.cpp - dynamic linking, [RansomActivity_processDonation]) without checking for bounds.

The maximum value for a signed 32-bit integer is 2,147,483,647.

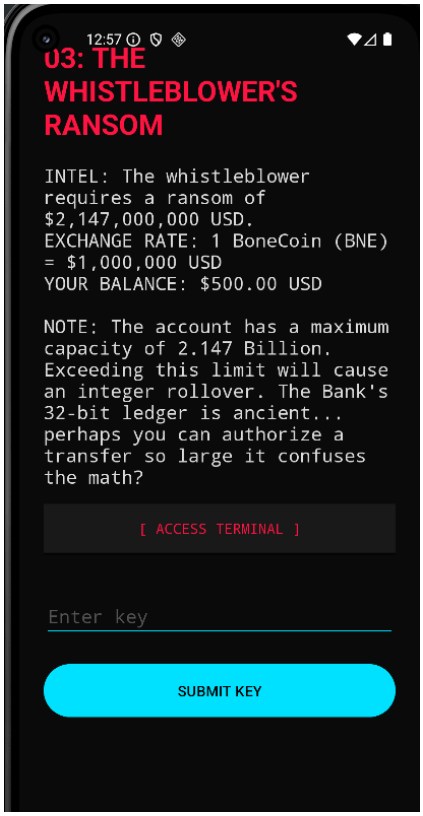
To flip the "sign bit" (making the number negative), we need a total cost greater than the max integer.

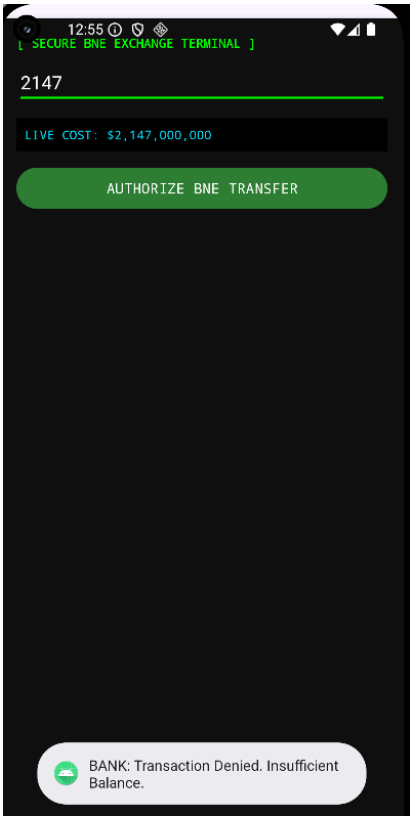
- 2147 x 1,000,000 = 2,147,000,000$ (Still positive, fail).
- 2148 x 1,000,000 = 2,148,000,000$ (Overflows to -2,146,967,296).

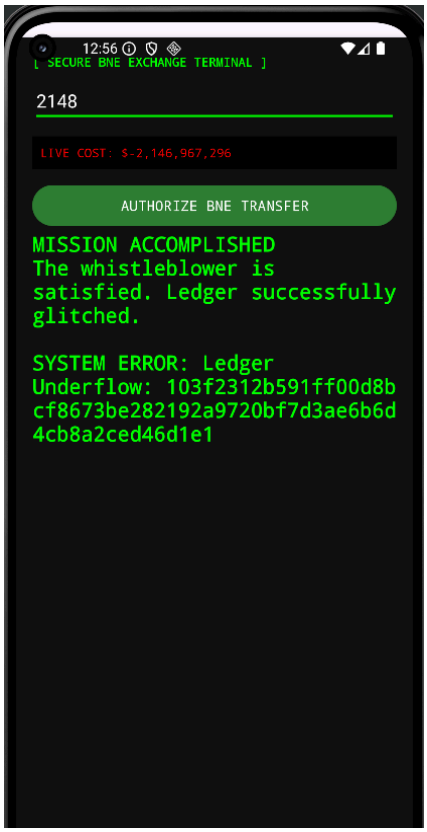
> The logic check (Negative Number <= 500) returns True, authorizing a transaction for billions of units despite having nearly zero funds.

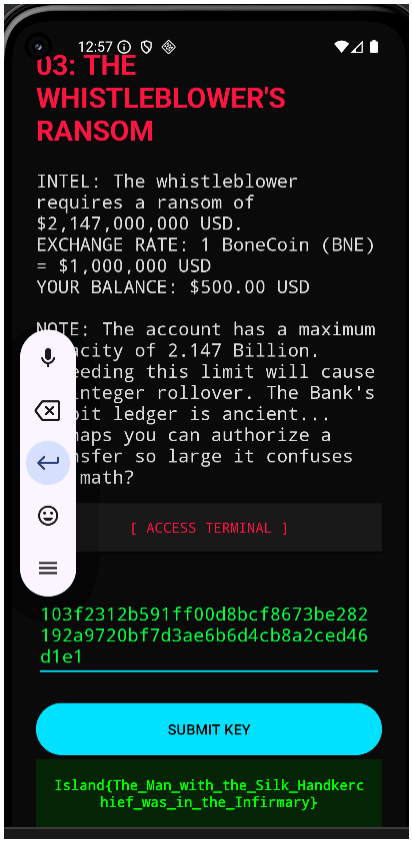
The logic check (Negative Number <= 500) returns True, authorizing a transaction for billions of units despite having nearly zero funds.

Input Qty	Result	32-bit Binary Result	Outcome
5	$5,000,000$	Positive value.	$5M > 500$ (Denied)
2,148	$2,148,000,000$	10000000... (The far-left bit is now 1)	$-2,146,967,296 < 500$ (Success!)
9,000	$9,000,000,000$	00100001... (It wrapped so far it became positive again!)	$410M > 500$ (Denied)


12:57
U3: THE WHISTLEBLOWER'S RANSOM
INTEL: The whistleblower requires a ransom of $2,147,000,000 USD.
EXCHANGE RATE: 1 BoneCoin (BNE) = $1,000,000 USD
YOUR BALANCE: $500.00 USD
NOTE: The account has a maximum capacity of 2.147 Billion. Exceeding this limit will cause an integer rollover. The Bank's 32-bit ledger is ancient... perhaps you can authorize a transfer so large it confuses the math?
[ACCESS TERMINAL]
Enter key
SUBMIT KEY


12:55
[SECURE BNE EXCHANGE TERMINAL]
2147
LIVE COST: $2,147,000,000
AUTHORIZE BNE TRANSFER
BANK: Transaction Denied. Insufficient Balance.


12:56
[SECURE BNE EXCHANGE TERMINAL]
2148
LIVE COST: $-2,146,967,296
AUTHORIZE BNE TRANSFER
MISSION ACCOMPLISHED
The whistleblower is satisfied. Ledger successfully glitched.
SYSTEM ERROR: Ledger Underflow: 103f2312b591ff00d8bcf8673be282192a9720bf7d3ae6b6d4cb8a2ced46d1e1


12:57
U3: THE WHISTLEBLOWER'S RANSOM
INTEL: The whistleblower requires a ransom of $2,147,000,000 USD.
EXCHANGE RATE: 1 BoneCoin (BNE) = $1,000,000 USD
YOUR BALANCE: $500.00 USD
[ACCESS TERMINAL]
103f2312b591ff00d8bcf8673be282192a9720bf7d3ae6b6d4cb8a2ced46d1e1
SUBMIT KEY
Island{The_Man_with_the_Silk_Handkerchief_was_in_the_Infirmary}

4. PRIVILEGED LOCATION **[Medium]**

   Move to a privileged location to get this clue. But wait! Only moving won't help. You have to get the right permission, only then can the Feds give you information!

Solution

```
public void checkSystemPrivilege() {
    TextView textView = (TextView) findViewById(R.id.tvStatusBox);
    textView.setVisibility(0);
    if (getApplicationContext().checkSelfPermission("android.permission.READ_PRIVILEGED_PHONE_STATE") == 0) {
        textView.setText("Feds have verified you! Here is the key to get the flag" + reception());
        textView.setTextColor(Color.parseColor("#00FF41"));
    } else {
        textView.setText("FEDS : Access denied!");
        textView.setTextColor(Color.parseColor("#00FF41"));
    }
}
```

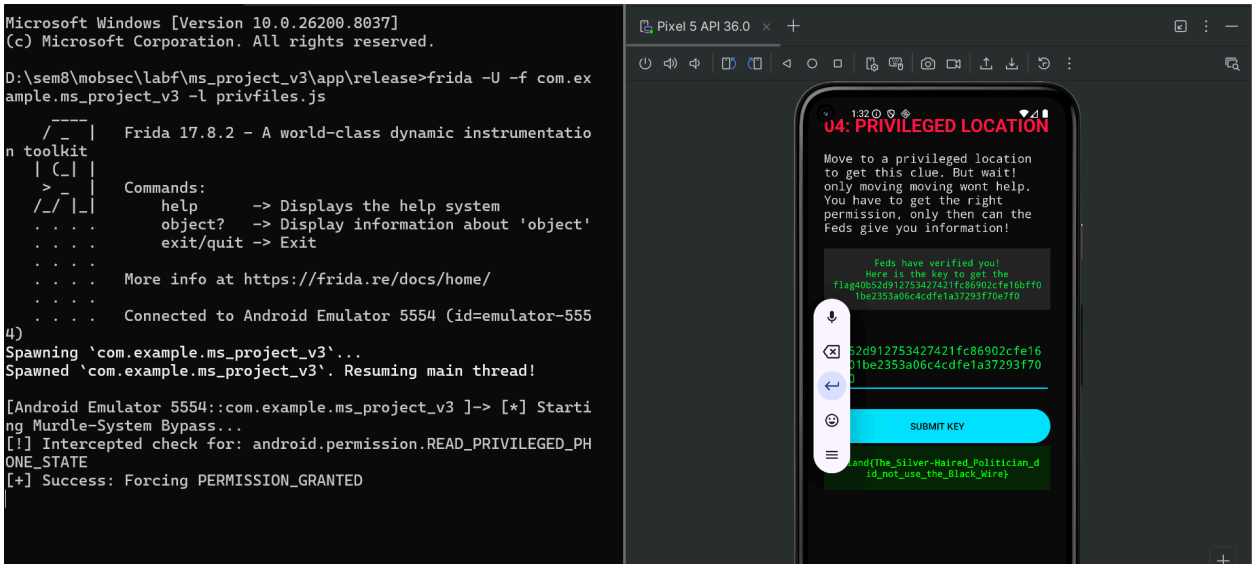
The key is provided if READ_PRIVILEGED_PHONE_STATE permission is granted. However this permission is not requested in the manifest file

Method -1 (frida hooking)

```
Java.perform(function () {
  console.log("[*] Starting Murdle-System Bypass...");

  var ContextWrapper = Java.use('android.content.ContextWrapper');
  var PackageManager = Java.use('android.content.pm.PackageManager');

  ContextWrapper.checkSelfPermission.implementation = function (permission) {
    if (permission === "android.permission.READ_PRIVILEGED_PHONE_STATE") {
      console.log("[!] Intercepted check for: " + permission);
      console.log("[+] Success: Forcing PERMISSION_GRANTED");
      return 0; // 0 is PackageManager.PERMISSION_GRANTED
    }
    return this.checkSelfPermission(permission);
  };
```



Method -2 (Using magisk)

5. CELL BLOCK PROXY **[EASY]**

The 'Public Messenger' is efficient but dangerously trusting. It accepts any parcel, no matter who it's from, and delivers it to whatever place is written on the label. It doesn't realize it’s being used as a proxy to open doors that should stay locked. If you can give the right 'Target Room' to it ,it’ll lead you straight into the hiding place which has the evidence.

[Broadcast Intent Redirection. Look at AndroidManifest.xml to see if anything is interesting]

Solution: Intent Redirection. In Android, a Broadcast Receiver can act as a proxy.

In AndroidManifest.xml, the EvidenceVaultActivity is set to android:exported="false". This means no other app and no external user can open it directly.

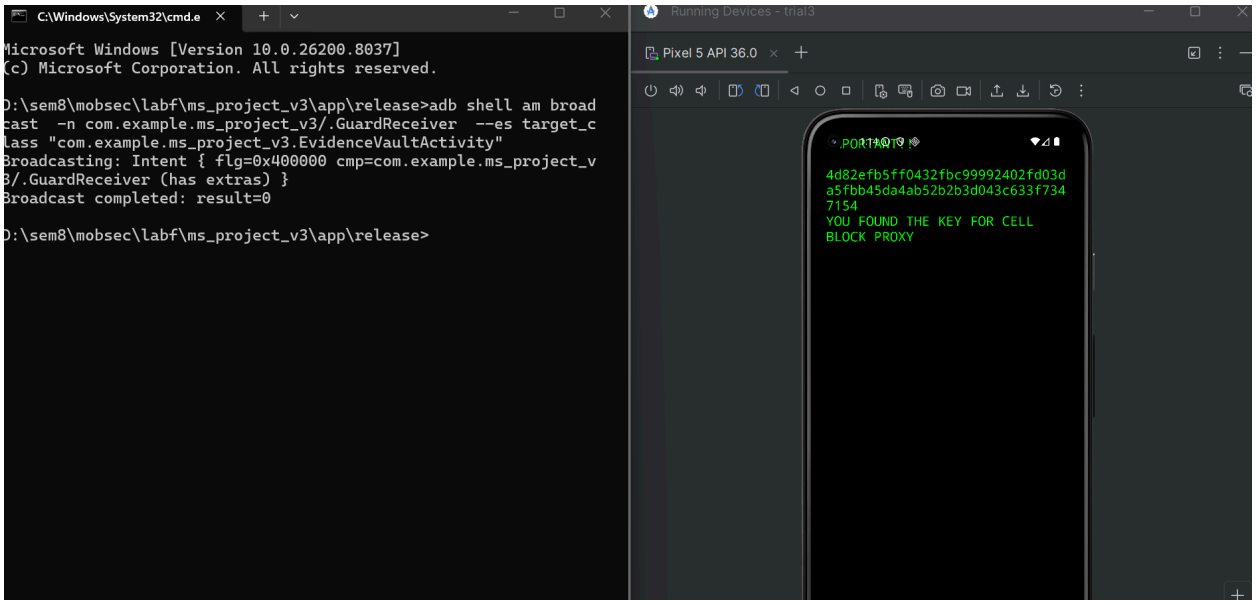
There is GuardReceiver which is exported="true". This means anyone (even another app or a user via ADB) can send a "message" (an Intent) to this Receiver.

```
<activity android:name="com.example.ms_project_v3.RansomActivity"/>
<activity android:name="com.example.ms_project_v3.NetworkActivity"/>
<activity android:name="com.example.ms_project_v3.HookerActivity"/>
<receiver
    android:name="com.example.ms_project_v3.GuardReceiver"
    android:exported="true"/>
<activity
    android:name="com.example.ms_project_v3.EvidenceVaultActivity"
    android:exported="false"/>
<activity
```

```
public void onReceive(Context context, Intent intent) {
    String stringExtra = intent.getStringExtra("target_class");
    Toast.makeText(context, "Guard received: " + stringExtra, 0).show();
    if (stringExtra != null) {
        try {
            Intent intent2 = new Intent();
            intent2.setClassName(context.getPackageName(), stringExtra);
            intent2.addFlags(268435456);
            context.startActivity(intent2);
        } catch (Exception e) {
```

The GuardReceiver takes a string from the message called target_class and blindly uses it to start an activity. It doesn't check what class is being requested.

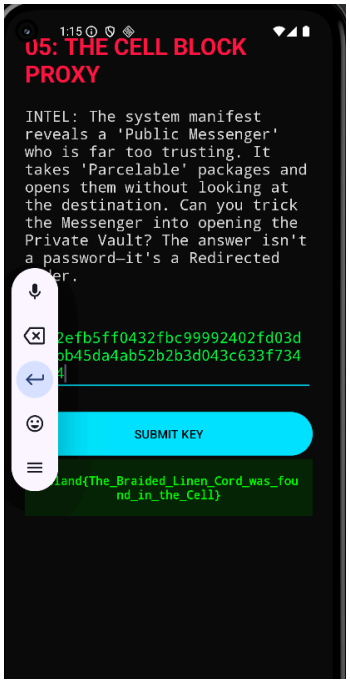
By sending a broadcast with the full path of the hidden activity (com.example.<package_name>.EvidenceVaultActivity), you can trick the app into launching a private screen.



- Receiver: com.example.<package_name>.GuardReceiver

- Hidden Activity: com.example.<package_name>.EvidenceVaultActivity
- Required Key: target_class

adb shell am broadcast -n com.example.ms_project_v3/.GuardReceiver --es target_class "com.example.ms_project_v3.EvidenceVaultActivity"



6. ARCHIVE DUMP **[EASY]**

   The Archive was supposed to be wiped, but the system's 'Mirroring Protocol' was too fast. While the glass screen shows you a 'Corruption Error,' the actual data was whispered into the shadows of the application's private ledger.

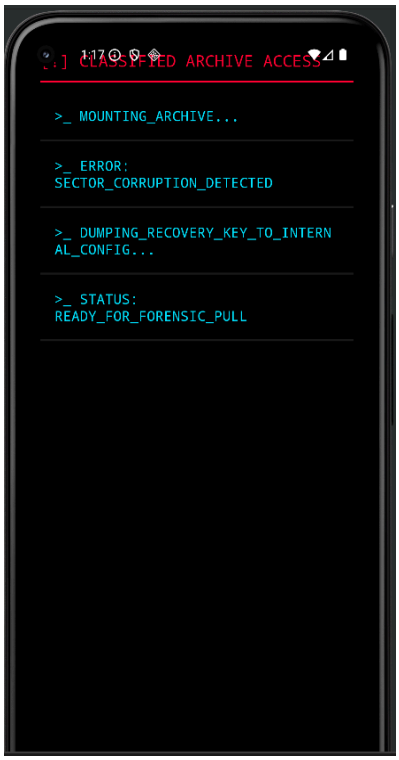
   [Insecure Data Storage in SharedPreferences]

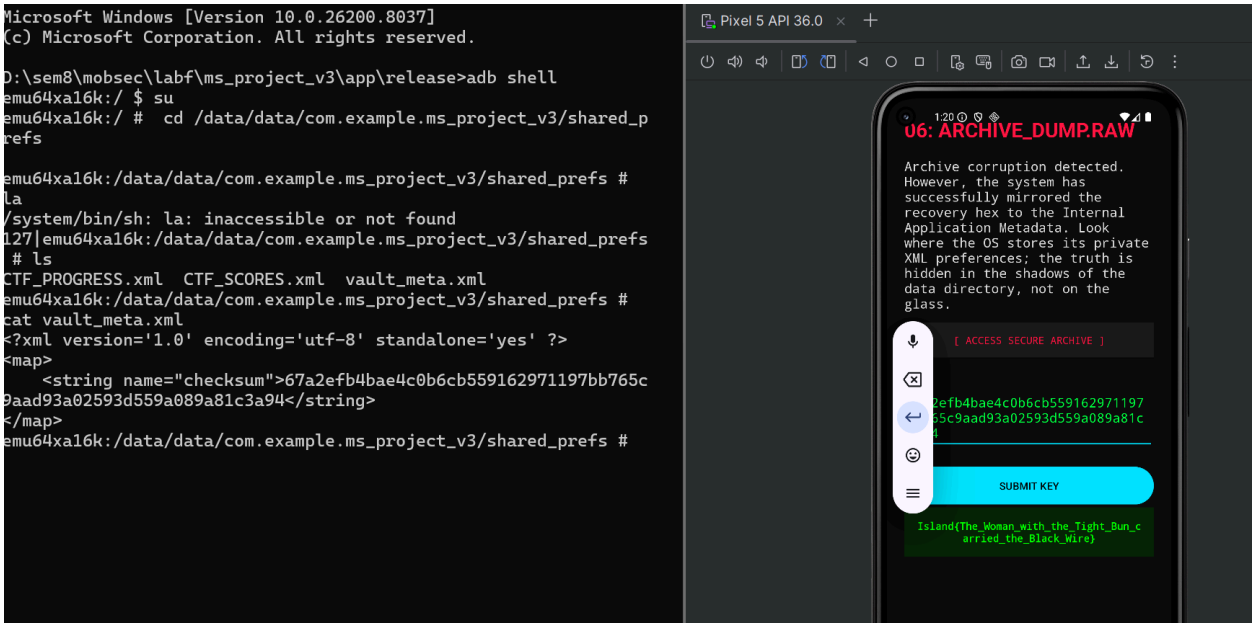
   Solution:

   Decompile the APK using JADX. Search for getSharedPreferences. You will find that the app writes to a file named vault_meta. The activity screen shows no flag. Looking at the source code for ArchiveActivity, we see the app uses getSharedPreferences("vault_meta", MODE_PRIVATE).

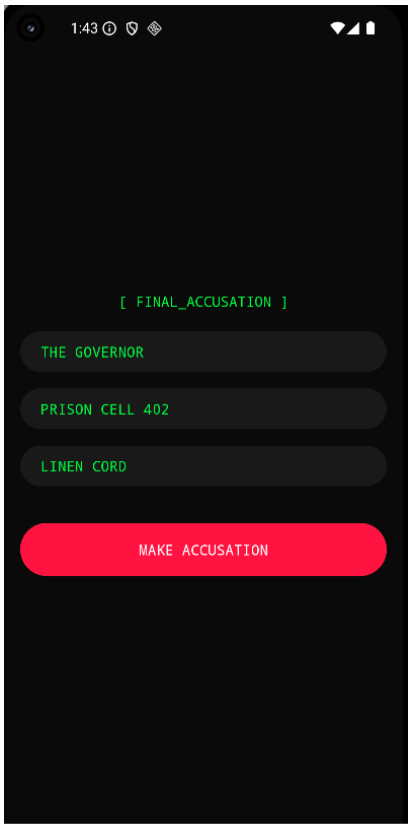
1. Enter Shell: adb shell

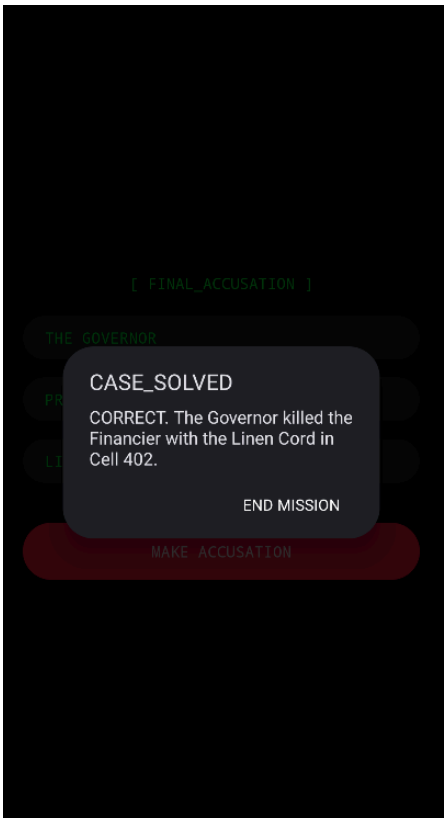
2. Switch to Root: su
3. Navigate: cd /data/data/cpm.example.ms_project_v3/shared_prefs/
4. Read: cat vault_meta.xml





ACCUSATION:


1:43
[FINAL_ACCUSATION]
THE GOVERNOR
PRISON CELL 402
LINEN CORD
MAKE ACCUSATION


[FINAL_ACCUSATION]
THE GOVERNOR
CASE_SOLVED
CORRECT. The Governor killed the Financier with the Linen Cord in Cell 402.
END MISSION
MAKE ACCUSATION